HOTEL BOOKING SYSTEM

PROJECT TITLE

Hotel Booking System

PROBLEM STATEMENT

Managing hotel room bookings manually is inefficient and error-prone.
This system automates room availability, booking, and cancellation.

TARGET USER

Hotel receptionists and small hotel staff who need a simple system to manage room bookings efficiently.

SYSTEM DESIGN

High-Level Architecture

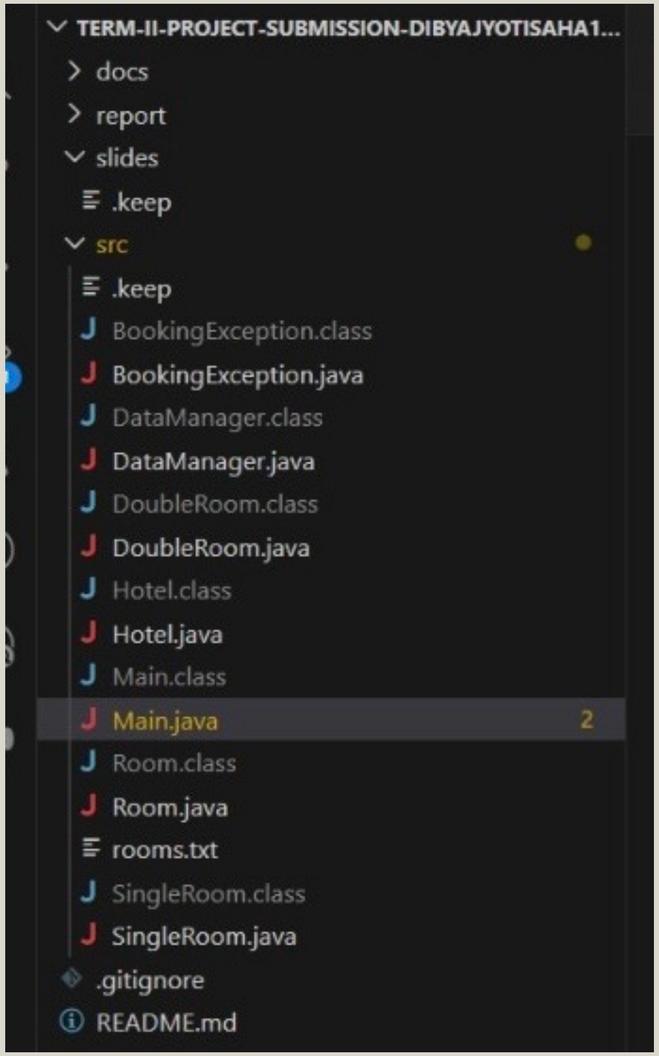
The system follows a modular object-oriented design:

- Main handles user interaction
- Hotel manages all operations
- Room defines the base structure
- SingleRoom and DoubleRoom represent specific room types
- DataManager handles file storage
- BookingException manages errors

Key Classes and Responsibilities

- Room (Abstract Class)
- Defines common attributes such as room number, price, and booking status
- SingleRoom and DoubleRoom
- Extend Room and implement specific behavior
- Hotel
- Manages room collection and handles booking and cancellation
- DataManager
- Saves and loads room data from file
- BookingException
- Handles invalid operations
- Main
- Provides menu-driven interface and controls program flow

PACKAGE STRUCTURE



OOP CONCEPTS USED

Abstraction

- **Implemented using the abstract class Room**
- **Defines common structure (room number, price, booking status)**
- **Hides implementation details from the user**

Inheritance

- **SingleRoom and DoubleRoom extend Room**
- **Reuse common code and add specific behavior**
- **Reduces code duplication**

Polymorphism

- **Achieved using List<Room> in the Hotel class**
- **Different room types are treated uniformly as Room**
- **Method displayDetails() behaves differently based on object type**

TECHNICAL HIGHLIGHT

Key Design Decision

Use of a common collection List<Room> to manage all room types along with file-based data persistence using DataManager.

Explanation of Logic

All room objects (SingleRoom, DoubleRoom) are stored in a single list using the base type Room. This allows uniform handling of different room types while still executing their specific behaviors through polymorphism. Additionally, room data is saved to a file and reloaded when the application starts, ensuring that booking information is not lost between runs.

Why This Approach Was Chosen

- Enables scalability by allowing new room types to be added without modifying existing logic
- Simplifies management by using a single collection instead of multiple structures
- Improves flexibility through polymorphism
- Adds real-world usability by preserving booking data using file handling
- Keeps the system modular and easy to maintain

SHOW WORKING SYSTEM

```
PS C:\Users\SK Enterprise\OneDrive\Desktop\term-ii-project-submission-dibyajyotisaha18-cmyk\src> javac Main.java
PS C:\Users\SK Enterprise\OneDrive\Desktop\term-ii-project-submission-dibyajyotisaha18-cmyk\src> java Main

HOTEL BOOKING SYSTEM:
1. View Rooms
2. Book Room
3. Cancel Booking
4. Exit
Enter choice: 1
Single Room | Room No: 101 | Price: 1000.0 | Booked: false
Single Room | Room No: 102 | Price: 1000.0 | Booked: false
Double Room | Room No: 201 | Price: 2000.0 | Booked: true
Double Room | Room No: 202 | Price: 2000.0 | Booked: false

HOTEL BOOKING SYSTEM:
1. View Rooms
2. Book Room
3. Cancel Booking
4. Exit
Enter choice: 2
Enter room number to book: 201
Error: Room already booked!

HOTEL BOOKING SYSTEM:
1. View Rooms
2. Book Room
3. Cancel Booking
4. Exit
Enter choice: 3
Enter room number to cancel: 201
Booking cancelled successfully!
```

Demonstration Plan

1. Start the Application

Run the program and show the menu-driven interface.

2. View Available Rooms

Select option 1 to display all rooms with details such as room number, type, price, and booking status.

3. Book a Room (Normal Case)

Select option 2 and enter a valid room number (for example, 101).

Show that the room status changes to booked.

4. Verify Booking

Again select option 1 to confirm that the selected room is now marked as booked.

Error Handling Demonstration

5. Attempt to Book the Same Room Again

Select option 2 and enter the same room number.

Show that the system prevents double booking and displays an error message.

6. Cancel Booking

Select option 3 and enter the same room number to cancel the booking.

Show that the room becomes available again.

7. Invalid Input Case

Try entering an invalid room number (for example, 999).

Show that the system handles the error gracefully with an appropriate message.

THANK YOU